
大数据与人工智能
自动化学院

Chapter02-Python基础

Python使用

- 推荐使用Python 3.x版本
- 建议通过Anaconda/Miniconda安装
- 核心数据科学库：NumPy和Matplotlib

基本语法特点

- 动态类型语言，变量类型自动推断
- 使用缩进表示代码块
- 支持列表、字典等数据结构

接下来我们将介绍大数据、大语言模型和相关大数据的操作。

不过在这之前，本章将简单地介绍一下Python，看一下它的使用方法。

2.1 Python是什么

- Python是一个简单、易读、易记的编程语言，而且是开源的，可以免费地自由使用。
- Python可以用类似英语的语法编写程序，编译起来也不费力，因此我们可以很轻松地使用Python。
- 事实上，很多高校和大专院校的计算机课程均采用Python作为入门语言。
- 使用Python不仅可以写出可读性高的代码，还可以写出性能高（处理速度快）的代码。

- 实际上， Google、 Microsoft、 Facebook等战斗在IT行业最前沿的企业也经常使用Python。
- 在科学领域，特别是在机器学习、数据科学领域， Python被大量使用。
- 深度学习的框架中也有很多支持使用Python

PyTorch	**是 (原生)**	由 Meta (Facebook) 开发，以其灵活性、易用性和强大的社区支持而闻名。命令式编程风格（动态计算图）使其非常适合研究和快速原型设计。是目前学术界和许多工业应用中的首选。
TensorFlow	**是 (原生)**	由 Google 开发，是第一个被广泛采用的深度学习框架。以其强大的生态系统、可扩展性和生产部署能力（如 TensorFlow Serving）而著称。TensorFlow 2.x 默认使用 Keras 作为其高级 API，也支持动态图。
JAX	**是 (原生)**	由 Google DeepMind 开发，是一个用于高性能机器学习研究的新兴框架。它结合了 `Autograd`（自动微分）和 `XLA`（加速线性代数），可以对原生 Python 和 NumPy 代码进行求导和编译，在GPU/TPU上实现极高的性能。
Keras	**是 (原生)**	一个高级神经网络 API，设计时考虑了用户友好性和快速实验。它现在是 TensorFlow 的官方高级 API (`tf.keras`)，但也可以作为独立库与其他后端（如 Theano 或 JAX）配合使用。非常适合初学者和快速搭建模型。
MXNet	**是**	由 Apache 基金会维护，以其可扩展性和对多种语言的支持而闻名。它允许混合使用命令式和符号式编程，性能出色，被亚马逊云科技 (AWS) 大力支持。

综上， Python是最适合数据科学领域的编程语言之一。而且， Python具有 受众广的优秀品质，从初学者到专业人士都在使用。

完成本课程的大数据与人工智能的目标， Python可以说是最合适的工具

2.2 Python的安装

下面，我们首先将Python安装到当前环境（电脑）上。这里说明一下安装时需要注意的一些地方。

2.2.1 Python版本

- Python有Python 2.x和Python 3.x两个版本。目前在人工智能领域的Python的以3.x为主，旧的版本2.x尽量不要使用(有的语法不兼容)
- 因为两个版本之间没有兼容性（严格地讲，是没有“向后兼容性”），也就是说，会发生用Python 3.x写的代码不能被Python 2.x执行的情况。
- 本课程使用Python 3.10

2.2.2 使用的外部库

- 本章主要使用NumPy库和Matplotlib库。
- NumPy是用于数值计算的库，提供了很多高级的数学算法和便利的数组（矩阵）操作方法
- Matplotlib是用来画图的库。使用Matplotlib能将实验结果可视化，并在视觉上确认深度学习运行期间的数据。

2.2.3 Python安装与包管理

- **Anaconda vs. Miniconda： Python环境与包管理**

  核心概念
- **共同点**: 两者都是由 Anaconda, Inc. 公司开发的 Python 环境管理工具。它们的核心都是 conda——一个强大的跨平台包管理器和环境管理器。
- **目标**: 解决 Python 包的安装、依赖冲突以及创建隔离的开发环境等问题。

核心工具	conda	conda
初始大小	**非常大 (GBs)**	**非常小 (MBs)**
预装包	**720+ (数据科学全家桶)**	**极少 (仅基础依赖)**
管理界面	包含 Anaconda Navigator (GUI)	仅命令行 (CLI)
目标用户	初学者、希望一键配好环境的用户	中高级用户、希望环境干净可控的开发者

2.3 Python解释器

- 推荐在Linux(比如Ubuntu, CentOS)下安装
- 完成Python的安装后，要先确认一下Python的版本。打开终端，输入 python --version命令，该命令会输出已经安装的Python的版本信息。

```
python --version
```

如果是在jupyter notebook里面，运行：

```
!python --version
```

说明已正确安装了Python 3.x。接着输入python，启动Python解释器。

mini conda安装

使用 wget 命令从官方源下载 Miniconda 安装脚本：

```
wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh
```

安装mini conda

```
bash Miniconda3-latest-Linux-x86_64.sh
```



启动conda环境

```
source ~/miniconda3/bin/activate
conda init
```

验证安装

```
conda list
```



Python环境配置和安装

📌 雨课堂问答：关于Python 2.x与3.x版本的关系

1. 创建 Python 3.10 环境

首先，使用 conda 命令创建一个新的环境，并指定 Python 版本为 3.10。

```
conda create --name bigai python=3.10
```

2. 激活新环境

创建环境后，您需要激活它：

```
conda activate bigai
```

激活后，终端提示符将显示当前激活的环境名称。

3. 安装所需的包（可选）

在激活的环境中，您可以安装项目所需的任何额外 Python 包。例如，如果您的项目需要 numpy 和 pandas，可以使用以下命令：

```
conda install numpy pandas
```



Python解释器也被称为“对话模式”，用户能够以和Python对话的方式 进行编程

比如，当用户询问“1 + 2等于几？”的时候， Python解释器会回答“3”，所谓对话模式，就是指这样的交互。现在，我们实际输入一下看看。

```
(bigai) [root@CentOS-PVE ~]# python
Python 3.10.4 (main, Mar 31 2022, 08:41:55) [GCC 7.5.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> 1+2
3
>>>
```

2.3.1 算术计算加法或乘法等算术计算，可按如下方式进行。

```
>>> 1 - 2
-1
>>> 4 * 5
20
```

```
>>> 7 / 5
1.4
>>> 3 ** 2
9
```

*表示乘法， /表示除法， **表示乘方（3**2是 3 的 2 次方）。

- 另外，在Python 2.x中，整数除以整数的结果是整数，比如， 7 ÷ 5的结果是1
- 但在Python 3.x中，整数除以整数的结果是小数（浮点数)

2.3.2 数据类型

- 编程中有数据类型（data type）这一概念。
- 数据类型表示数据的性质，有整数、小数、字符串等类型。
- Python中的 type()函数可以用来查看数据类型。

```
>>> type(10)
<class 'int'>
>>> type(2.718)
<class 'float'>
>>> type("hello")
<class 'str'>
```

根据上面的结果可知， 10是int类型（整型）， 2.718是float类型（浮点型），"hello"是 str（字符串）类型。

- 另外，“类型”和“类”这两个词有时用作相同的意思。
- 这里，对于输出结果<class 'int'>，可以将其解释成“10是int类（类型）”。

2.3.3 变量

- 直接使用x或y等字母定义变量，不需要先声明。
- 可以使用变量进行计算。
- 可以对变量赋值。

```
>>> x = 10 # 初始化
>>> print(x) # 输出x
10
>>> x = 100 # 赋值
>>> print(x)
100
>>> y = 3.14
>>> x * y
314.0
>>> type(x * y)
<class 'float'>
```

- Python是属于“动态类型语言”的编程语言，所谓动态，是指变量的类 型是根据情况自动决定的。
- 在上面的例子中，用户并没有明确指出“x的类型是 int（整型）”
- Python根据 x被初始化为10，从而判断出 x的类型为int的
- 我们也可以看到，整数和小数相乘的结果是小数（数据类型的自动转换）
- “#”是注释的意思，它后面的文字会被Python忽略

1.3.4 列表

- 除了单一的数值，还可以用列表（数组）汇总数据。

```
>>> a = [1, 2, 3, 4, 5] # 生成列表
>>> print(a) # 输出列表的内容
[1, 2, 3, 4, 5]
```

```
>>> len(a)  # 获取列表的长度
5
>>> a[0]  # 访问第一个元素的值
1
>>> a[4]
5
>>> a[4] = 99 # 赋值
>>> print(a)
[1, 2, 3, 4, 99]
```

- 元素的访问是通过a[0]这样的方式进行的。
- []中的数字称为索引（下标）
- 索引区间严格遵循“左闭右开”**原则。用数学符号表示为 [start, end)，即**包含起始索引（左边界），但不包含结束索引（右边界）。
- 索引从0开始（索引0对应第一个元素）。

```
>>> print(a)
[1, 2, 3, 4, 99]
>>> a[0:2] # 获取索引为0到2（不包括2！）的元素
[1, 2]
>>> a[1:] # 获取从索引为1的元素到最后一个元素
[2, 3, 4, 99]
>>> a[:3] # 获取从第一个元素到索引为3（不包括3！）的元素
[1, 2, 3]
```

- Python的列表提供了切片（sIicing）这一便捷的标记法。使用切片不仅可以访问某个值，还可以访问列表的子列表（部分列表）。
- 元素的访问是通过 a[0] 这样的方式进行的。[] 中的数字称为**索引**（下标），索引从 0 开始（索引 0 对应第一个元素）。
- 另外，Python 支持**负数索引**：索引 -1 对应最后一个元素，-2 对应倒数第二个元素，以此类推。

```
>>> a[:-1] # 获取从第一个元素到最后一个元素的前一个元素之间的元素
[1, 2, 3, 4]
>>> a[:-2] # 获取从第一个元素到最后一个元素的前二个元素之间的元素
[1, 2, 3]
```

1.3.5 字典

- 列表根据索引，按照0, 1, 2, . . .的顺序存储值
- 而字典则以键值对的形式存储数据。
- 字典就像《新华字典》那样，将单词和它的含义对应着存储起来。

```
>>> me = {'height':180} # 生成字典
>>> me['height']  # 访问元素
180
>>> me['weight'] = 70
>>> print(me)
{'height': 180, 'weight': 70}
```

字典在深度学习 (PyTorch) 中的实战

- **字典的本质**：以“键-值”对应存储，是深度学习框架底层的核心数据结构之一。
- **应用场景**：集中管理训练超参数（Hyperparameters），让实验配置清晰且易于修改。

```
import torch
```

```python
# 1. 构建与管理训练超参数字典
config = {
    'batch_size': 64,
    'learning_rate': 0.01,
    'optimizer': 'Adam'
}

# 动态更新超参数（例如学习率衰减）
config['learning_rate'] = 0.005



# 2. 揭秘 PyTorch 的模型权重字典（模拟 state_dict）
# 键（Key）是网络层的名称，值（Value）是包含权重的张量（Tensor）
model_weights = {
    'conv1.weight': torch.randn(16, 3, 3, 3), # 随机初始化卷积核张量
    'fc1.bias': torch.zeros(10)               # 将全连接层偏置设为0
}

# 通过键名，精准访问对应网络层的权重
print("全连接层偏置大小:", model_weights['fc1.bias'].shape)
```

1.3.6 布尔型

- bool型取True或False中的一个值。
- 针对 bool 型的运算符包括 and、or 和 not（称为逻辑运算符）；
- 针对数值型的运算符包括 +、-、*、/ 等（称为算术运算符）。
- 根据不同的数据类型使用不同的运算符）。

```
>>> hungry = True  # 饿了?
>>> sleepy = False # 困了?
>>> type(hungry)
<class 'bool'>
>>> not hungry
False
>>> hungry and sleepy # 饿并且困
False
>>> hungry or sleepy # 饿或者困
True
```

📌 雨课堂问答：在Python中，变量的类型是如何确定的?

1.3.7 if语句

- 根据不同的条件选择不同的处理分支时可以使用if/else语句。

```
>>> hungry = True
>>> if hungry:
...     print("I'm hungry")
I'm hungry
>>> hungry = False
>>> if hungry:
...     print("I'm hungry") # 使用空白字符进行缩进
... else:
...     print("I'm not hungry")
...     print("I'm sleepy")
I'm not hungry
I'm sleepy
```

- Python中的空白字符具有重要的意义。
- 上面的if语句中， if hungry:下面的语句开头有4个空白字符。它是缩进的意思，表示当前面的条件（if hungry） 成立时，此处的代码会被执行。
- 这个缩进也可以用tab表示， Python中推荐使用空白字符。
- Python使用空白字符表示缩进。一般而言，每缩进一次，使用4个空白字符。
- 注意：控制语句后面的冒号“:”必须有

1.3.8 for 语句

- 进行循环处理时可以使用for语句。

```
>>> for i in [1, 2, 3]:
... print(i)
...
1 2 3
```

- 这是输出列表[1, 2, 3]中的元素的例子。
- 使用for ... in ... :语句结构，可以按顺序访问列表等数据集合中的各个元素。

- for常常搭配range()函数使用

```
>>> for i in range(3):
...  print(i)
...
0
1
2
```

列表推导式(List Comprehension)，是Python中用来基于现有列表快速生成新列表的精炼写法。

```
[新元素表达式 for 变量 in 原集合]
```

以 `[x + 1 for x in py_list]` 为例，它的执行逻辑包含三步：

1. **遍历**：`for x in py_list` 负责依次从原列表中取出每个元素赋值给 `x`。
2. **计算**：最左侧的 `x + 1` 对取出的元素进行加工计算。
3. **收集**：外层的方括号 `[...]` 自动将每次计算得到的新结果收集起来，拼装成一个全新的列表。

与普通 for 循环的等价对比

```
[x + 1 for x in py_list]
```

它完全等同于以下传统的 3 行代码，但免去了手动创建空列表和调用 .append() 的繁琐步骤：

```
py_list_result = []
for x in py_list:
    py_list_result.append(x + 1)
```

核心优势

- **精炼易读**：将“映射/变换”的意图浓缩为单行表达式。
- **执行更快**：列表推导式在底层由 C 语言循环优化执行，运行速度略快于 Python 显式调用 append()。

📌 雨课堂问答：关于Python中的列表推导式

1.3.9 函数

- 可以将一连串的处理定义成函数（function）。

```
>>> def hello():
... print("Hello World!")
...
>>> hello()
Hello World!
```

- 此外，函数可以导入参数。

```
>>> def hello(object):
... print("Hello " + object + "!")
...
>>> hello("cat")
Hello cat!
```

- 另外，字符串的拼接可以使用+。

关闭Python解释器

- 输入exit()，可以退出
- 如果使用快捷键——
    - Linux或Mac OS X的情况下输入Ctrl-D（按住Ctrl，再按D键）
    - Windows的情况下输入Ctrl-Z，然后按Enter键。

2.4 Python脚本文件 (Python代码文件)

- 到目前为止，我们看到的都是基于Python解释器的例子。Python解释器能够以对话模式执行程序，非常便于进行简单的实验。
- 想进行一连串的处理时，因为每次都需要输入程序，所以不太方便。
- 这时，可以将Python程序保存为文件，然后（集中地）运行这个文件。
- 下面，我们来看一个Python脚本文件的例子

2.4.1 保存为文件

- 打开文本编辑器，新建一个 hungry.py的文件。 hungry.py只包含下面一行语句

```
print("I'm hungry!")
```

- 接着，打开终端（Windows中的命令行窗口），移至hungry.py所在的位置。
- 然后，将 hungry.py文件名作为参数，运行 python命令。

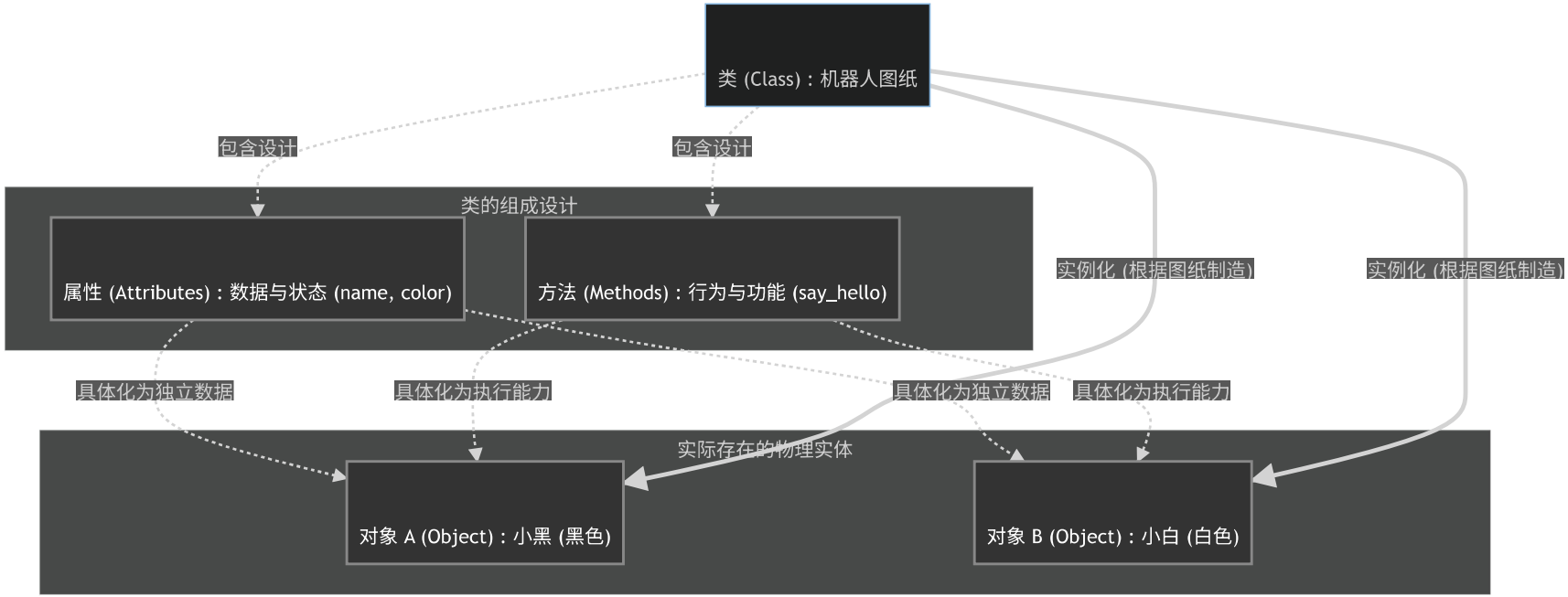
```
python hungry.py
```

I'm hungry!

- 这样，使用python hungry.py命令就可以执行这个Python程序了。

2.4.2 类

- 前面 int和str等数据类型（通过 type()函数可以查看对象的类型）。
- 这些数据类型是“内置”的数据类型，是Python中一开始就有的数据类型。
- 现在，我们来定义新的类。如果用户自己定义类的话，就可以自己创建数据类型。
- 此外，也可以定义原创的方法（类的函数）和属性。


类 (Class)：机器人图纸
包含设计
包含设计
类的组成设计
属性 (Attributes)：数据与状态 (name, color)
方法 (Methods)：行为与功能 (say_hello)
实例化 (根据图纸制造)
实例化 (根据图纸制造)
具体化为独立数据
具体化为执行能力
具体化为独立数据
具体化为执行能力
实际存在的物理实体
对象 A (Object)：小黑 (黑色)
对象 B (Object)：小白 (白色)

- 类的定义模板

```
class 类名:
def __init__(self, 参数, …): # 构造
函数
...
def 方法名1(self, 参数, …):  # 方法1
...
def 方法名2(self, 参数, …):  # 方法2
...
```

[redacted]__init__方法，这是进行初始化的方法，也称为构造函数（constructor）
,只在生成类的实例时被调用一次。可以进行属性数据的初始化

- 此外，在方法的第一个参数中明确地写入表示自身（自身的实例）的self是

Python的一个特点

```python
# 1. 定义类（画图纸）
class Robot:
    # 属性：在初始化方法 __init__ 中赋予特征
    def __init__(self, name, color):
        self.name = name    # 属性：名字
        self.color = color  # 属性：颜色

    # 方法：定义它可以执行的动作
    def say_hello(self):
        print(f"你好，我是{self.color}的机器人，我叫{self.name}！")

# 2. 实例化对象（根据图纸造机器）
robot_a = Robot("小黑", "黑色")
robot_b = Robot("小白", "白色")

# 3. 调用方法（让机器动起来）
robot_a.say_hello()  # 输出：你好，我是黑色的机器人，我叫小黑！
robot_b.say_hello()  # 输出：你好，我是白色的机器人，我叫小白！
```

把上面代码保存为MAIN.PY，从终端运行MAN.PY。

```
$ python main.py


你好，我是黑色的机器人，我叫小黑！
你好，我是白色的机器人，我叫小白！
```

- 这里我们定义了一个新类Robot。
- 类Robot生成了实例（对象） robot_a, robot_b。
- 类 Robot的构造函数（初始化方法）会接收参数name, color，然后用这个参数初始化实例变量 self.name, self.color。
- 实例变量是存储在各个实例中的变量。
- Python中可以像self.name, self.name这样，通过在self后面添加属性名来生成或访问实例变量

2.5 NumPy

- 在深度学习的实现中，经常出现数组和矩阵的计算。
- NumPy的数组类（numpy.array）中提供了很多便捷的方法，在实现深度学习时，我们将使用这些方法。

本节我们来简单介绍一下后面会用到的NumPy

2.5.1 导入NumPy

- NumPy是外部库。这里所说的“外部”是指不包含在标准版Python中。
  因此，我们首先要导入NumPy库。

```
>>> import numpy as np
```

- Python 中使用 import语句来导入库。
- 这里的 import numpy as np，直译的话就是“将numpy作为np导入”的意思。
- 通过写成这样的形式，之后NumPy相关的方法均可通过np来调用。

2.5.2 生成NumPy数组

要生成NumPy数组，需要使用np.array()方法。 np.array()接收Python列表作为参数，生成NumPy数组（类型是numpy.ndarray）。

```
>>> x = np.array([1.0, 2.0, 3.0])
>>> print(x)
[ 1. 2. 3.]
>>> type(x)
<class 'numpy.ndarray'>
```

2.5.3 NumPy 的算术运算

下面是NumPy数组的算术运算的例子。

```
>>> x = np.array([1.0, 2.0, 3.0])
>>> y = np.array([2.0, 4.0, 6.0])
>>> x + y # 对应元素的加法
array([ 3., 6., 9.])
>>> x - y
array([ -1., -2., -3.])
>>> x * y # element-wise product
array([ 2., 8., 18.])
>>> x / y
array([ 0.5, 0.5, 0.5])
```

这里需要注意的是，数组x和数组y的元素个数是相同的（两者均是元素个数为3的一维数组）。

- 当 x和 y的元素个数相同时，**对应的元素**进行算术运算。
- 如果元素个数不同，程序就会报错，所以元素个数保持一致非常重要。
  另外，“对应元素的”的英文是element-wise，比如“对应元素的乘法”就是element-wise product

- NumPy数组不仅可以进行element-wise运算，也可以和单一的数值（标量）组合起来进行运算。
- 此时，需要在NumPy数组的各个元素和标量之间进行运算。
- 这个功能也被称为广播（详见后文）。

```
>>> x = np.array([1.0, 2.0, 3.0])
>>> x / 2.0
array([ 0.5, 1. , 1.5])
```

- 在 Python 中，**列表 (List)** 和 **NumPy 数组 (ndarray)** 都是用于存储数据集合的结构，但它们的设计初衷和底层机制有着本质的区别。
- 打个比方：Python 列表就像是生活中的“杂物收纳箱”，**什么都能装，非常灵活；而 NumPy 数组则像是工业港口的**“标准化集装箱”，只能装同一种货物，但在大批量装卸（数值计算）时效率极高。

核心对比分析

数据类型	**混合**：可以同时包含整数、字符串、甚至其他列表。	**单一**：所有元素必须是同一种类型（如纯浮点数），这被称为“同质性”。
内存结构	**分散存储**：存储的是对象的“指针”（内存地址），元素本身散落在内存各处。	**连续存储**：在内存中开辟一整块连续空间，底层采用 C 语言级别的数组实现。
计算速度	**较慢**：每次操作都需要经过 Python 解释器去判断数据类型。	**极快**：利用连续内存和底层 C 语言优化（以及 CPU 级别的 SIMD 指令），速度可达列表的百倍以上。
数学运算	缺乏原生数学支持。做加法 `[1, 2] + [3, 4]` 的结果是拼接：`[1, 2, 3, 4]`。	支持**向量化（Vectorization）**和广播机制。做加法是对应位置相加。
适用场景	通用编程、收集不同类型的数据、频繁增删元素的场景。	科学计算、深度学习（张量 Tensor 的前身）、大规模矩阵与图像处理。

简单示例：对集合中的每个数字加1，效果对比



```
用 Python 列表
================================
, 2, 3, 4]

1 会报错：TypeError
py_list + 1

r 循环或列表推导式逐个提取、计算、再组合
lt = [x + 1 for x in py_list]
n 列表结果:", py_list_result)  # 输出: [2, 3, 4, 5]



================================
用 NumPy 数组
================================
p.array([1, 2, 3, 4])

 直接对整个数组进行数学运算，底层自动应用到每一个元素
ult = np_array + 1
 数组结果:", np_array_result)  # 输出: [2 3 4 5]
```

代码解析： NumPy 彻底省略了 for 循环。这种“将操作直接应用于整个数据集合”的特性称为向量化，它是机器学习和数据科学中处理海量数据时，代码既简洁又高效的基石。

2.5.4 NumPy的N维数组

NumPy不仅可以生成一维数组（排成一列的数组），也可以生成多维数组。比如，可以生成如下的二维数组（矩阵）。

```
>>> A = np.array([[1, 2], [3, 4]])
>>> print(A)
[[1 2]
[3 4]]
>>> A.shape
(2, 2)
>>> A.dtype
dtype('int64')
```

- 上面生成了一个2 × 2的矩阵A。
- 矩阵A的形状可以通过shape查看
- 矩阵元素的数据类型可以通过dtype查看。

下面，我们来看一下矩阵的算术运算。

```
>>> B = np.array([[3, 0],[0, 6]])
>>> A + B
array([[ 4, 2],
[ 3, 10]])
>>> A * B
array([[ 3, 0],
[ 0, 24]])
```

- 和数组的算术运算一样，矩阵的算术运算也可以在相同形状的矩阵间以对应元素的方式进行。
- 并且，也可以通过标量（单一数值）对矩阵进行算术运算。
    - 这也是基于广播的功能。

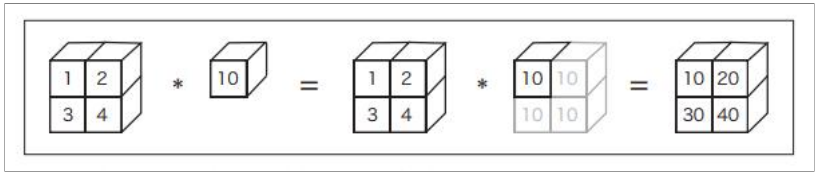
```
>>> print(A)
[[1 2]
[3 4]]
>>> A * 10
array([[ 10, 20],
[ 30, 40]])
```

- NumPy 数组（ np.array）可以生成 N 维数组，即可以生成一维数组、二维数组、三维数组等任意维数的数组。
- 数学上将一维数组称为向量， 将二维数组称为矩阵。
- 可以将一般化之后的向量或矩阵等统称为张量（tensor）。

📌 雨课堂问答：关于Python中的numpy数组运算

2.5.5 广播

- NumPy中，形状不同的数组之间也可以进行运算。之前的例子中，在2× 2的矩阵A和标量10之间进行了乘法运算。
- 在这个过程中，如图所示，标量 10被扩展成了2 × 2的形状，然后再与矩阵 A进行乘法运算。

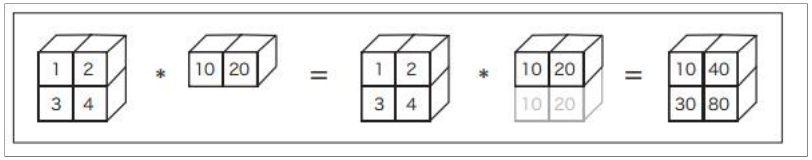


- 这个自动扩展计算的功能称为广播（broadcast）

我们通过下面这个运算再来看一个广播的例子。

```
>>> A = np.array([[1, 2], [3, 4]])
>>> B = np.array([10, 20])
>>> A * B
array([[ 10, 40],
[ 30, 80]])
```

- 在这个运算中，如图所示，一维数组 B被“巧妙地”变成了和二维数组A相同的形状，



- 然后再以对应元素的方式进行运算。
  综上，因为NumPy有广播功能，所以不同形状的数组之间也可以顺利地进行运算。

2.5.6 访问元素

- 元素的索引从0开始。对各个元素的访问可按如下方式进行。

```
>>> X = np.array([[51, 55], [14, 19], [0, 4]])
>>> print(X)
[[51 55]
[14 19]
[ 0 4]]
>>> X[0] # 第0行
array([51, 55])
>>> X[0][1] # (0,1)的元素
55
```

- 也可以使用for语句访问各个元素。

```
>>> for row in X:
...     print(row)
...
[51 55]
[14 19]
[0 4]
```

- 除了前面介绍的索引操作， NumPy还可以使用数组访问各个元素。

```
>>> X = X.flatten() # 将X转换为一维数组
>>> print(X)
[51 55 14 19 0 4]
>>> X[np.array([0, 2, 4])] # 获取索引为0、 2、 4的元素
array([51, 14, 0])
```

运用这个标记法，可以获取满足一定条件的元素。例如，要从 X中抽出大于15的元素，可以写成如下形式。

```
>>> X > 15
array([ True, True, False, True, False, False], dtype=bool)
>>> X[X>15]
array([51, 55, 19])
```

对NumPy数组使用不等号运算符等（上例中是 X > 15），结果会得到一个布尔型的数组。 上例中就是使用这个布尔型数组取出了数组的各个元素（取出True对应的元素）。

补充：

- Python等动态类型语言一般比C和C++等静态类型语言（编译型语言）运算速度慢。
- 实际上，如果是运算量大的处理对象，用C/C++写程序更好。为此，当 Python 中追求性能时，人们会用 C/C++ 来实现处理的内容。
- Python 则承担“中间人”的角色，负责调用那些用 C/C++写的程序。
- NumPy中，主要的计算处理也都是通过C或C++实现的。
  因此，我们可以在不损失性能的情况下，使用Python便利的语法。

2.6 Matplotlib

- 在大数据分析中，图形的绘制和数据的可视化非常重要。
- Matplotlib是用于绘制图形的库，使用Matplotlib可以轻松地绘制图形和实现数据的可视化。
- 这里介绍一下图形的绘制方法和图像的显示方法。

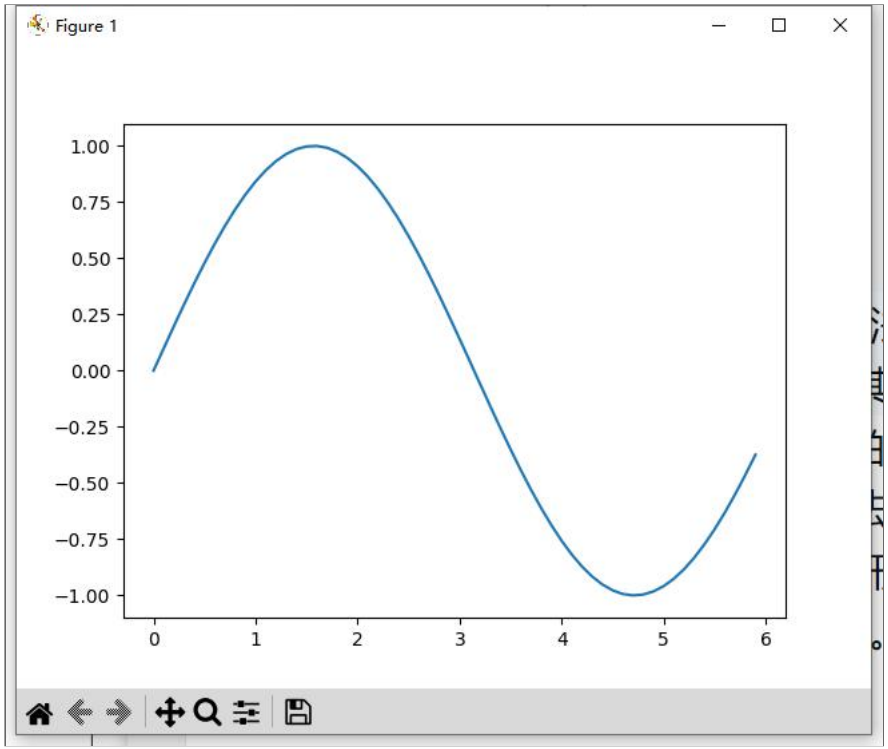
2.6.1 绘制简单图形

- 可以使用 matplotlib的 pyplot模块绘制图形。
- 话不多说，我们来看一个绘制sin函数曲线的例子。

```
import numpy as np
import matplotlib.pyplot as plt
# 生成数据
x = np.arange(0, 6, 0.1) # 以0.1为单位，生成0到6的数据
y = np.sin(x)
# 绘制图形
plt.plot(x, y)
plt.show()
```

- 这里使用NumPy的 arange方法生成了 [0, 0.1, 0.2, ..., 5.8, 5.9]的数据，将其设为x。
- 对x的各个元素，应用NumPy的sin函数np.sin()，将x、y的数据传给plt.plot方法，然后绘制图形。
- 最后，通过plt.show()显示图形。

运行上述代码后，就会显示图形。

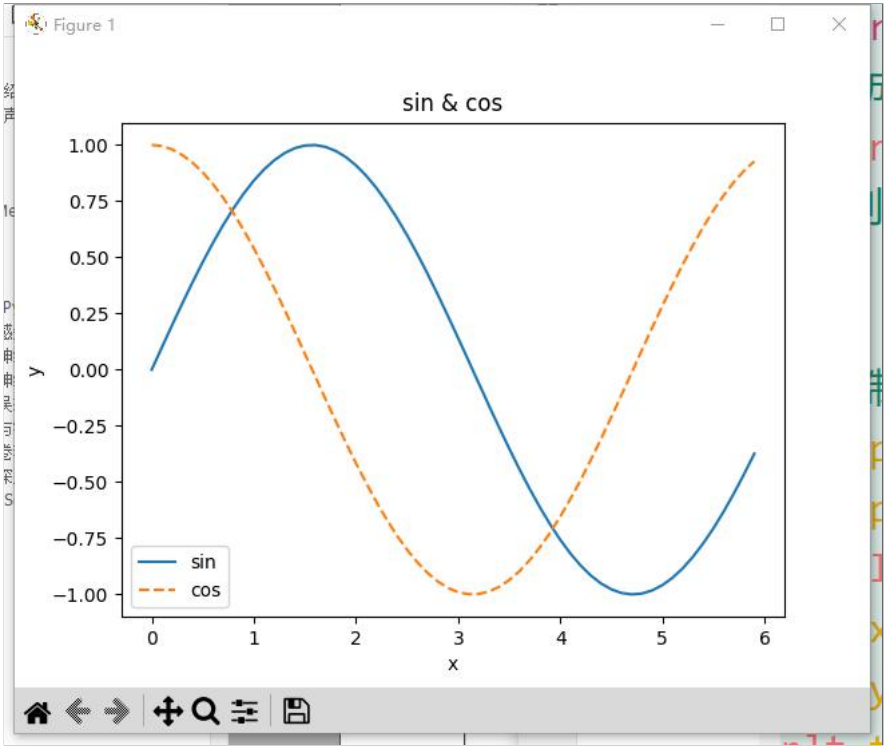


2.6.2 pyplot的功能

- 在刚才的sin函数的图形中，我们继续追加cos函数的图形
- 并且尝试使用pyplot的添加标题和x轴标签名等其他功能。

```
import numpy as np
import matplotlib.pyplot as plt
# 生成数据
x = np.arange(0, 6, 0.1) # 以0.1为单位，生成0到6的数据
y1 = np.sin(x)
y2 = np.cos(x)
# 绘制图形
plt.plot(x, y1, label="sin")
plt.plot(x, y2, linestyle = "--", label="cos") # 用虚
线绘制
plt.xlabel("x") # x轴标签
plt.ylabel("y") # y轴标签
plt.title('sin & cos') # 标题
plt.legend()
plt.show()
```

结果如图所示，我们看到图的标题、轴的标签名都被标出来了。



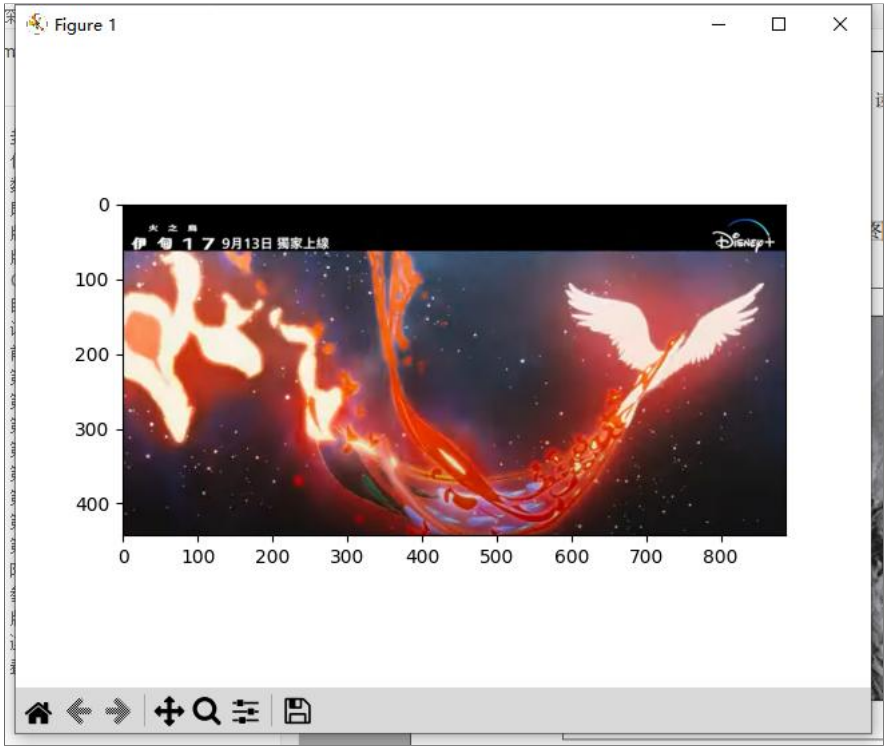
2.6.3 显示图像

- pyplot 中还提供了用于显示图像的方法 imshow()。
- 可以使用matplotlib.image模块的imread()方法读入图像 下面我们来看一个例子。

```
import matplotlib.pyplot as plt
from matplotlib.image import imread
img = imread('lena.png') # 读入图像（设定合适的路径！）
plt.imshow(img)
plt.show()
```

运行上述代码后，会显示图1-5所示的图像。

- 这里，我们假定图像lena.png在当前目录下。
- 根据自己的环境，可能需要变更文件名或文件路径。
- 比如，'../dataset/Phoenix.png'，即可正确运行



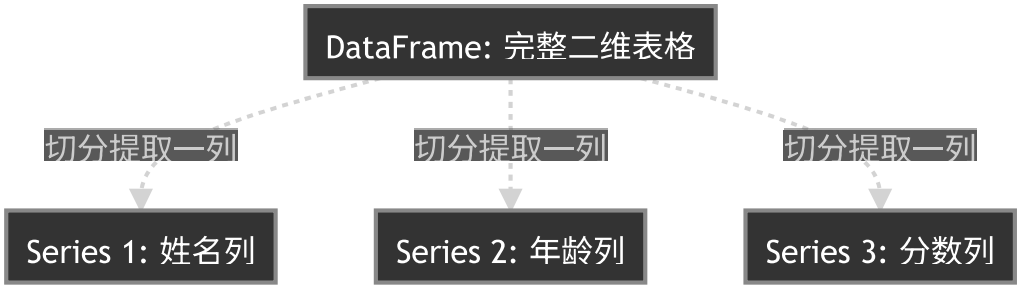
2.7.1 认识 Pandas： Python 里的超级 Excel

如果说 NumPy 负责纯数字的矩阵运算，那么 **Pandas** 就是处理表格型数据的终极利器。

- 它就像代码版的 Excel，能轻松清洗、筛选、统计和分析杂乱的实际业务数据。
- **两大核心武器**：
    - **Series**：一维带标签数组（相当于表格的一单列）。
    - **DataFrame**：二维二维表格（行和列组成的整张表）。

2.7.2. Pandas 的数据结构全景

DataFrame 是由多个 Series 拼接而成的整表，包含明确的行索引（Index）与列名（Columns）。



2.7.3 创建第一个数据表

使用字典即可秒建 DataFrame，Pandas 会自动生成行索引（0, 1, 2...）与列名：

```
import pandas as pd

# 使用字典定义表格数据
data = {
    "姓名": ["张三", "李四", "王五"],
    "年龄": [20, 22, 21],
    "成绩": [85, 92, 78]
}

# 转换成 DataFrame 表格
df = pd.DataFrame(data)
print(df)
```

2.7.4 数据的读取与提取

Pandas 能够一键读取本地文件，并像字典一样快速调取单列或定位行：

```
# 1. 常见文件读取（支持 Excel、CSV 等）
# df = pd.read_csv("data.csv")

# 2. 提取整列（返回一个 Series）
ages = df["年龄"]

# 3. 按标签提取某行（例如第一行）
first_row = df.loc[0]

print("年龄列平均值:", ages.mean())
```

2.7.5 条件筛选与清洗实战

无需编写任何复杂的 for 循环，直接使用逻辑表达式即可筛选符合条件的行：

```
# 筛选成绩大于等于 80 的所有学生记录
high_scores = df[df["成绩"] >= 80]
print(high_scores)

# 快速查看数据的统计特征（均值、最值等）
print(df.describe())
```

- **总结**：熟练掌握 Pandas 是进入数据分析与机器学习特征工程的必经之路。

2.8 小结

本章重点介绍了实现大数据&人工智能所需的Python编程知识

- Python是一种简单易记的编程语言。
- Python是开源的，可以自由使用。
- 本书中将使用Python 3.x实现深度学习。
- 本书中将使用NumPy和Matplotlib这两种外部库。
- Python有“解释器”和“脚本文件”两种运行模式。
- Python能够将一系列处理集成为函数或类等模块。
- NumPy中有很多用于操作多维数组的便捷方法。

- 从下一章开始，我们将通过使用Python实际运行代码，逐步了解深度学习。
- 本章只介绍了关于Python的最低限度的知识，想进一步了解Python，可以参考下面图书。


TURING 图灵程序设计丛书
no starch press
Python Crash Course
A Hands-On, Project-Based Introduction to Programming
Python编程
从入门到实践
【美】Eric Matthes 著 袁国忠 译